

WINGBREAKER

UAV detector & interceptor

A fixed-wing interceptor that patrols a box, sees an intruder drone on a gimbal camera, chases and orbits it, then fires only after a human — or an optional LLM — says yes.



01	THE PROBLEM — WHY A DETECTOR THAT CANNOT CHASE IS NOT A SYSTEM
02	ARCHITECTURE — LAYERS, NODES, AND THE CONTRACTS BETWEEN THEM
03	THE SOLUTION — PATROL, LOCK, DECIDE, ENGAGE, REPORT
04	OPTIONAL · THE OPERATOR LOOP
05	OPTIONAL · SAFETY AS A SECOND VOICE
06	OPTIONAL · THE WORLD IT FLIES IN
07	OPTIONAL · HONEST LIMITS AND WHAT COMES NEXT

Wingbreaker is a complete simulated interceptor loop: perception, mission supervision, flight control, a safety veto, an approval gate, and an operator dashboard — launched with one command. Two ROS 2 packages: `wingbreaker_interfaces` (messages, FlyToGPS, interception service) and `wingbreaker_uav` (every node, launch, dashboard). Sim assets live in `sim/`. Weights: `yolo26n_DroneDetection.pt` at the repo root.

01 The problem

Small drones are cheap, quiet, and already inside the fence before anyone looks up. Watching a camera feed is not a defence. A waypoint loop is not a defence. A detector that publishes boxes and then shrugs is not a defence.

The hard part is not “run YOLO.” The hard part is closing the loop from pixel to chase to a deliberate, auditable decision — without coupling the mission brain to PX4, Gazebo, or a particular LLM.

WHAT BREAKS IN THE WILD

- Perception stacks that never command the vehicle.
- Autopilots that fly a box with no idea what they are looking at.
- Ground stations that require a human to notice, decide, and click in time.
- Single-link telemetry: when MAVSDK stalls, safety stalls with it.

WHAT THIS PROJECT INSISTS ON

- One launch file brings up sim, flight, vision, and UI.
- The brain never talks MAVLink. It talks actions and services.
- Engagement is gated. Default mode is a human on the dashboard.
- Battery and airborne state can veto a chase independently.

The operational question

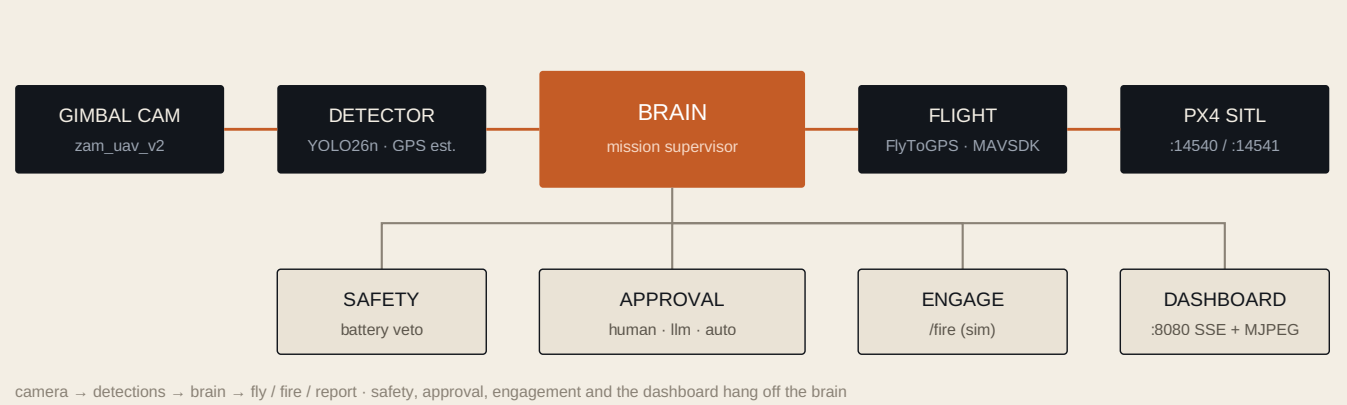
An interceptor sitting over a box has four jobs, in order: stay on station, notice a non-cooperative aircraft, get close enough to act, and only then spend the shot. Each job has a different failure mode. Miss the detection and you patrol forever. Detect without a lock and you write a log. Lock without an approval gate and you have built a weapon, not a system. Approve without a report and you cannot tell anyone what happened.

Wingbreaker is the simulation of that full sentence — not a hardware program, not a real interceptor. It is the software spine you would want first: typed interfaces, a supervised flight gateway, a state machine you can read in one sitting, and an operator surface that makes the decision visible.

02 Architecture

Four layers. The mission brain sits in the middle and refuses to know how Gazebo, PX4, or the browser work.

OPERATOR	Web dashboard on :8080 — mission state, MJPEG camera, detections, Approve, intercept reports. Optional QGroundControl tab via noVNC.
MISSION	brain · detector · intercept_llm · engagement_node. State machine, YOLO, approval policy, simulated fire.
VEHICLE	flight_node and safety_node, each with its own DroneGateway / MAVSDK / gRPC port. FlyToGPS action. Dual UDP into PX4 (14540 / 14541).
WORLD	PX4 SITL airframe 4031 · Gazebo zam_uav_v2 · gimbal camera · ros_gz_bridge · kinematic intruder_x500 · runway_world.



Node graph

What `rqt_graph` should show once the camera gate opens. `engagement_node` looks isolated because it only exposes services — that is correct, not a missing publisher.

TOPIC / CONTRACT	FROM	TO	WHY IT EXISTS
<code>/camera/image_raw</code>	ros_gz_bridge	detector, dashboard	Gimbal pixels. Gate waits for real frames, not a publisher.
<code>/detections</code>	detector	brain, LLM, dashboard	Intruder lat/lon/alt, confidence, threat, ROI.
<code>/fly_to_gps</code>	brain	flight_node	Cancelable GPS goal. Feedback is remaining metres.

<code>/safe_to_fly</code>	safety_node	brain	Battery veto. Chase is refused below 20%.
<code>/request_interception</code>	dashboard / LLM / CLI	brain	Approve or deny. Ignored outside DECISION.
<code>/fire</code>	brain	engagement_node	Simulated shot. Returns success + message.
<code>/mission_state · / intercept_reports</code>	brain	dashboard	Operator truth. Who approved, where, whether it hit.

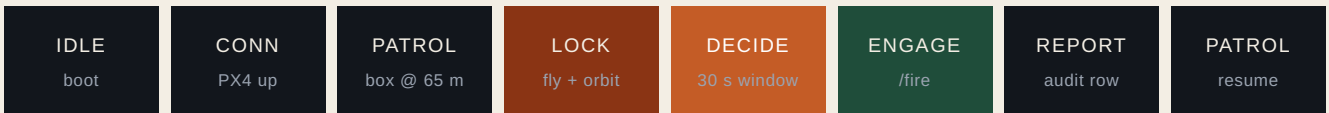
Interface package

The contracts are small on purpose. New behaviour should almost always be a new node talking these types, not a thicker message.

TYPE	NAME	PAYLOAD THAT MATTERS
msg	IntruderDetection	lat, lon, alt, confidence, threat_level, camera ROI
msg	MissionState	IDLE → CONNECTING → PATROL → LOCK → DECISION → ENGAGE → REPORT · ERROR
msg	VehicleStatus	connected, armed, airborne, pose, heading, battery, sats
msg	InterceptReport	detected, intercepted, pose, confidence, approved_by, message
action	FlyToGPS	goal lat/lon/alt · result success · feedback distance_remaining
srv	RequestInterception	approve, approver → success, message

03 The solution

One supervised state machine, a camera that estimates GPS from a bounding box, a flight node that actually arrives, and a fire service that cannot be called until someone — or a policy — approves.



Denied, timed out, or a failed lock all land in REPORT and cool down for 10 seconds before the next chase. The brain ignores detections while not in PATROL, while airborne is false, and while `safe_to_fly` is false.

See — detector

Two modes, one message. Live mode loads `yolo26n_DroneDetection.pt`, strides every third frame, keeps the strongest `drone` box above 0.35, and publishes only if confidence later clears the brain’s 0.55 chase threshold. Simulated mode (the original bootstrap) drops a synthetic intruder on a timer so the rest of the chain can be tested without weights.

Pixel-to-GPS is deliberately naive: the box centre is a bearing offset through the gimbal’s 114.6° HFOV, then a point `assume_range_m` (60 m) along that bearing from the vehicle. Altitude is copied from ownship relative altitude. Refine later with depth or target telemetry — do not pretend the current estimate is a tracker.

Go — flight and the gateway

`DroneGateway` owns one `asyncio` loop, one `MAVSDK System`, and a dedicated gRPC port. It reconnects forever. Telemetry (position, battery, armed, landed, heading, NED velocity, GPS) is pumped until the first stream dies, then the whole link is torn down. Callers never see frozen numbers from a dead socket.

Fixed-wing PX4 keeps `LandedState` `UNDEFINED` in cruise, so airborne is *landed-state OR relative altitude > 3 m*. Arrival is 100 m, not “a few metres”: L1 guidance loiters and will not converge like a quadrotor. Goals are cancelable; the brain cancels the current waypoint the instant a valid detection arrives.

Ask — three approval modes

MODE	WHO DECIDES	WHEN TO USE IT
human	Dashboard Approve, or CLI on <code>/request_interception</code>	Default. The honest operator-in-the-loop demo.
llm	<code>intercept_llm</code> node, same service, <code>approver="llm"</code>	Policy experiments. Mock provider ships; real APIs are a hook.
auto	Brain itself, immediately after lock	Closed-loop soak tests. Not the story you show first.

The LLM node is optional and never imported by the base system. Mock policy: approve if detection confidence ≥ 0.85 , after a 2 s think delay. OpenAI / Anthropic / Ollama raise `NotImplementedError` in `_ask_llm` on purpose — wiring a provider is a contained change, not a rewrite.

One command

```
BRING THE WHOLE STACK UP

ros2 launch wingbreaker_uav wingbreaker.launch.py
Options: intruder_mode:=static|flying|none · approval_mode:=human|llm|auto · run_llm:=true
Dashboard: http://localhost:8080 · QGC-in-browser (optional): qgc novnc.launch.py
```

04 The operator loop

Optional, but this is the piece that makes the system feel real. A supervisor that only logs to a terminal is a lab toy.

`web_dashboard` is a ROS 2 node with a stdlib `ThreadingHTTPServer`. No extra web framework. It serves one page, an SSE stream of live state, an MJPEG camera with the detection ROI overlaid, and a POST that calls `/request_interception` as `approver="human"`.

WHAT THE OPERATOR SEES

- Mission state, including “AWAITING APPROVAL”.
- Ownship: pose, heading, speed, battery, sats, airborne.
- Camera from the gimbal, with the YOLO box drawn on it.
- Every InterceptReport — detected, intercepted, who signed.
- A QGC tab if the noVNC bridge is listening on : 6080.

WHAT THE OPERATOR CAN DO

- Approve or walk away. Timeout is a deny.
- Keep flying QGC in the same browser if xvfb + x11vnc are installed.
- Leave the loop entirely: CLI service call still works.
- Swap the human for the LLM node without touching the brain.

The dashboard is adapted from the pothole-inspection pattern: one process, Server-Sent Events, no websocket library. Clients that fall behind are dropped rather than queued into a memory leak. That is the right trade for a local ops view, not a public site.

QGroundControl-in-browser is deliberately a sidecar (`novnc_bridge.py` + vendored noVNC), not a ROS node. Core sim does not depend on xvfb. If the operator wants a GCS tab, they launch it. If they do not, nothing is missing from the mission.

05 Safety as a second voice

Safety is not a function on the flight node. It is a second MAVSDK connection that can still speak if the primary one is wedged.

`flight_node` owns `udpin://0.0.0.0:14540` and gRPC 50051. `safety_node` owns `:14541` and gRPC 50052.

Dedicated ports exist because MAVSDK will otherwise reuse a stale `mavsdk_server` from the last crashed run and look “connected” while publishing ghosts.

Battery below 20% publishes `safe_to_fly = false`. The brain will not leave PATROL for a chase. Unknown battery (`-1`) is treated as safe so a missing telemetry field does not ground a sim that has not yet streamed percent — a pragmatic choice, documented here so nobody mistakes it for a flight-worthiness argument.

Two other gates sit in the brain itself. Detections during takeoff are ignored until `connected && airborne`. After a report, a cooldown swallows the next 10 seconds of boxes so the same intruder does not immediately re-trigger LOCK. Decision has a 30-second clock. Silence is a deny.

06 The world it flies in

The interceptor is a Reaper-styled fixed wing. The target is a scaled X500 that does not run a flight stack. That is a feature.

ASSET

ROLE

zam_uav_v2

Ownship. IMU, baro, mag, GPS, airspeed plugin, elevons/flaps/rudder, gimbal camera looking along the nose. Lidar removed — this mission is visual.

4031_gz_zam_uav_v2	PX4 airframe dropped into ROMFS. Launch sets <code>PX4_SYS_AUTOSTART=4031</code> and <code>PX4_SIM_MODEL=gz_zam_uav_v2</code> .
intruder_x500	Static-flagged, 3× PX4 X500 visual. Spawned and teleported with <code>gz service</code> . No physics, no companion computer.
runway_world	Default Gazebo world. Patrol waypoints sit on the stock PX4 Zurich spawn box (~47.398 N, 8.546 E).

Intruder modes: `flying` orbits several copies around the patrol box, `static` parks them at the orbit start, `none` leaves the sky empty. PX4 names the first vehicle `zam_uav_v2_0`, not `zam_uav_v2` — the camera bridge topic includes that suffix or the image never arrives.

Launch does not start the app nodes until `wait_for_topic` has seen an actual `sensor_msgs/Image` on `/camera/image_raw`. Checking for a publisher is not enough: `ros_gz_bridge` advertises immediately, even when Gazebo has nothing to forward. That single gate is why “full stack up” means the camera is alive, not merely that PX4 printed a banner.

TUNABLES LIVE IN ONE FILE

`wingbreaker_uav/config/uav.yaml` — waypoints, patrol altitude, both confidence thresholds, orbit radius (80 m), arrival radius (100 m), approval mode, YOLO path, dashboard port. ROS 2 forbids nested arrays, so waypoints are a flat `lat0, lon0, lat1, lon1, ...` list. Do not nest them.

07 Honest limits

A briefing that hides the seams is a brochure. These are the seams.

WHAT IS TRUE TODAY

- End-to-end verified: PATROL → LOCK → human Approve → ENGAGE → REPORT *intercepted=True* → resume.
- Live YOLO on the gimbal, not only the simulated detector.
- Supervised dual MAVSDK gateways that reconnect.
- Single-command launch with a data-based camera gate.

WHAT IS NOT

- Engagement is a service stub. There is no seeker, no warhead, no hit geometry.
- Range is assumed, not measured. A box on the horizon is still “60 m ahead.”
- The LLM path is a mock policy plus a hook.
- QGC-in-browser still needs `xvfb` and `x11vnc` on the host.

What I would do next

- **A real range.** Stereo, depth, or a second view so lock is a track, not a bearing plus a constant.
- **A real provider in `_ask_llm`.** Keep the mock. One live backend, same service, brain still does not know.
- **Multi-intruder arbitration.** Hold a track id; drop a stale box instead of always taking the strongest detection.
- **Hardware in the loop.** The gateway is the seam — swap SITL for a companion talking MAVSDK and the graph stays.